CURSOR PRACTICAL COURSE • SESSION 2 OF 5 • 90 min

Vibe coding

Build a memory game by only asking for something “nice”, then compare everyone's games at the end.



0:00 • everyone

Before we start

Before the lesson starts, everyone does these three things.

- [ ] Open `cursor-course/` and run `git pull`
- [ ] Leave the model on Auto (don't change the settings)
- [ ] Open HTML files by right-clicking in the sidebar → **Open In Browser**

If you missed session 1, finish the setup first, following chapter 1 of session 1.

LAST TIME

The loop hasn't changed

01
Ask
`@file + question`

Understand it

02
Agent
`@file + request`
`+ done-when`

Have it edit

03
diff
`read the change`

What actually changed

04
Keep / Undo
`keep it or drop it`

You decide

> What changes is **what you hand it**. The loop itself is the same today.

TODAY

What we do today

Today we only use the way of asking without deciding anything (**vibe coding**).

	What we do	Chapter
1	Build a memory game by asking “make it nice” without deciding anything	Ch. 2
2	Add the client's extra requests	Ch. 3
3	Change your game however you like	Ch. 4
4	Show everyone's games and compare them	Ch. 5

> In session 3 we write the requirements first and then build (**spec-driven development**).

TODAY'S RULES

Today's rules

01

Don't worry about rough edges

We ask carelessly on purpose

02

We don't compare how finished it is

We look at what is different

03

If you're stuck, raise your hand

Looking at your neighbour's screen is also quick

90 MINUTES

Timetable

Time	Chapter	What	Who
0:00	Ch. 1 Today's goal	What we do today (5 min)	Instructor
0:05	Ch. 2 Build it by vibe coding	Build a memory game with 3 lines (15 min)	All
0:20	**Ch. 3 A late request**	**Add the extra request to the game (10 min)**	All
0:30	Ch. 4 Change it freely	Add a feature you like (15 min)	All
0:45	**Ch. 5 Presentations**	**3 min each → compare together (20 min)**	All
1:05	Ch. 6 Summary	What vibe coding is like (10 min)	Instructor
1:15	Ch. 7 Review quiz and next time	Questions on sessions 1–2, next session (15 min)	All

> **Chapter 5 (presentations) is never cut.** Comparing everyone's games is how this session ends.

CURSOR PRACTICAL COURSE • SESSION 2 OF 5

CHAPTER 01 · 0:00 5 min

Today's goal

What happens when you ask without deciding anything. What we do today.

TODAY'S GOAL

Today's goal

1	**Build a working game by vibe coding**	Ch. 2
2	**Add a request and notice you can't check it went in correctly**	Ch. 3
3	**Change your game however you like**	Ch. 4
4	**Compare everyone's games in the presentations**	Ch. 5

> **All four happen within this session.** How finished the game is doesn't matter.

TERMS

Two words, used consistently

Word	What it points at
Vibe coding	The **way** of asking without deciding anything. Today we only do this
Spec-driven development	The **way** of writing requirements first, then asking. We do this in session 3

> **Both are names of ways of working.** The things we build are called by the game's name (“the memory game”). We don't shorten it to “vibe”.

CHAPTER 1 COLUMN

We'll run Agent a lot today

Thirty seconds on usage before we start.

Even on Auto	you're billed at **the price of whichever model it routed to**
Every plan	comes with some usage included
Once that's gone	you either pay as you go, or move up a plan

> Auto's settings (Cost / Balance / Intelligence) **change which model it routes to** — they aren't a discount. No prices in these slides. When it matters, check the official pricing page.

CURSOR PRACTICAL COURSE • SESSION 2 OF 5

CHAPTER 02 · 0:05 15 min

Build it by vibe coding

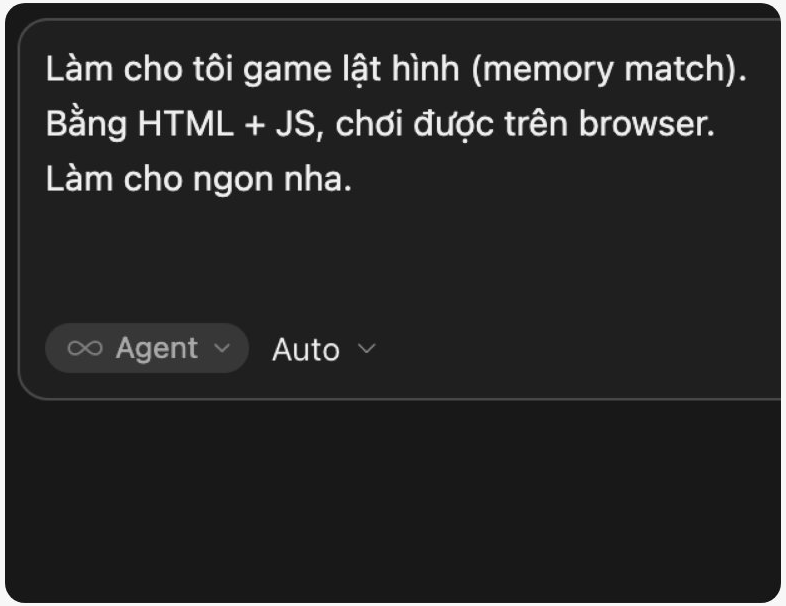
How far do you get without deciding anything? Everyone ends up with their own memory game.

CHAPTER 2 YOUR TURN

Have it build a memory game (12 min)

1. Make a new folder called `session02/` (1 min)
2. Open a new chat and send the three lines below (10 min)
3. When it's done, open it in the browser (1 min)
   Right-click in the sidebar → **Open In Browser**

```
Build me a memory match game.
HTML + JS, playable in the browser.
Make it nice.
```

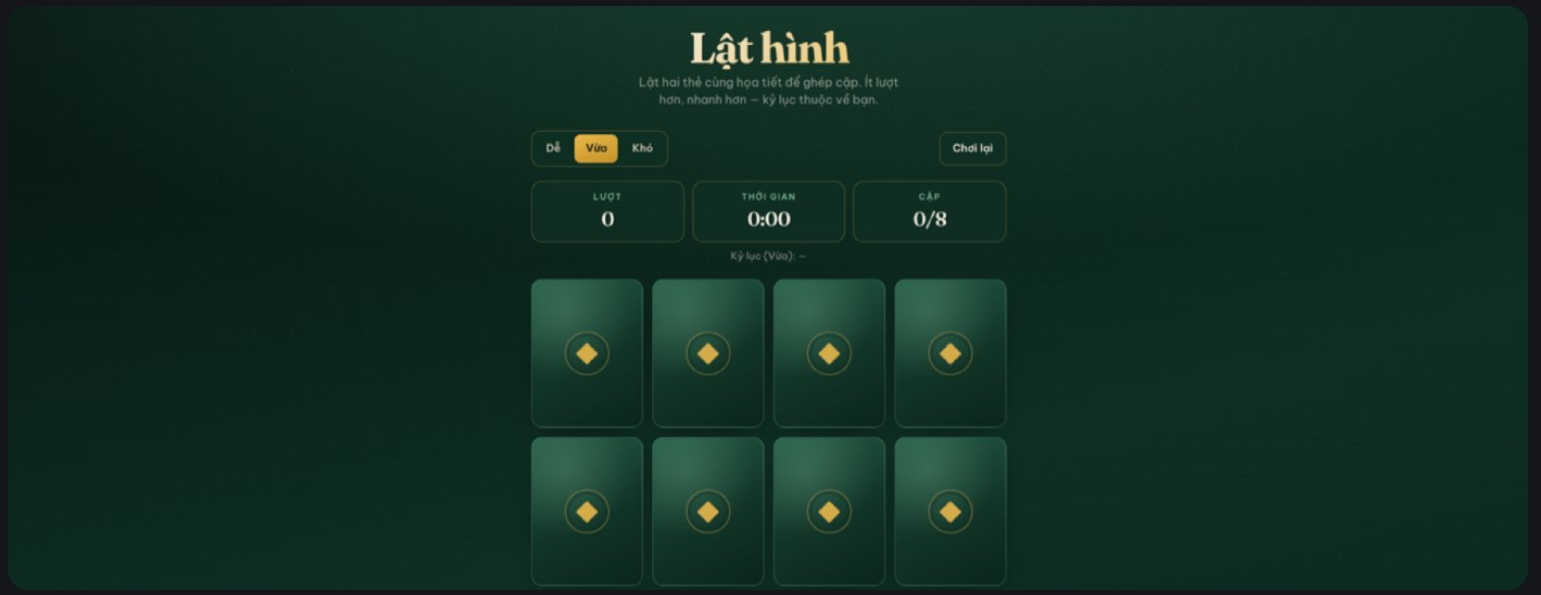


"Make it nice" is exactly the **phrasing we said to avoid** in session 1. **We're being sloppy on purpose.** Leave the model on **Auto**.

CHAPTER 2 SAMPLE

Minutes later, something runs

SAMPLE • yours will differ



> Fast. Something working, right away. **This much is vibe coding at its best.**

CHAPTER 2 IF YOU'RE STUCK

Not running? Have Agent look

Cursor's browser is **something Agent can drive**. Opening pages, clicking, even **reading errors out of the console**.

```
Open the game I just built in the browser,
and if there are console errors, tell me the cause.
```

Opening it in your own browser may not work. Open it in Cursor's built-in browser.

Browser actions need approval (by default it asks each time). In class, leave it as it is.

CURSOR PRACTICAL COURSE • SESSION 2 OF 5

CHAPTER 03 · 0:20 10 min

A late request

The client sends extra requests. What happens to the game you just built?

CHAPTER 3 YOUR TURN

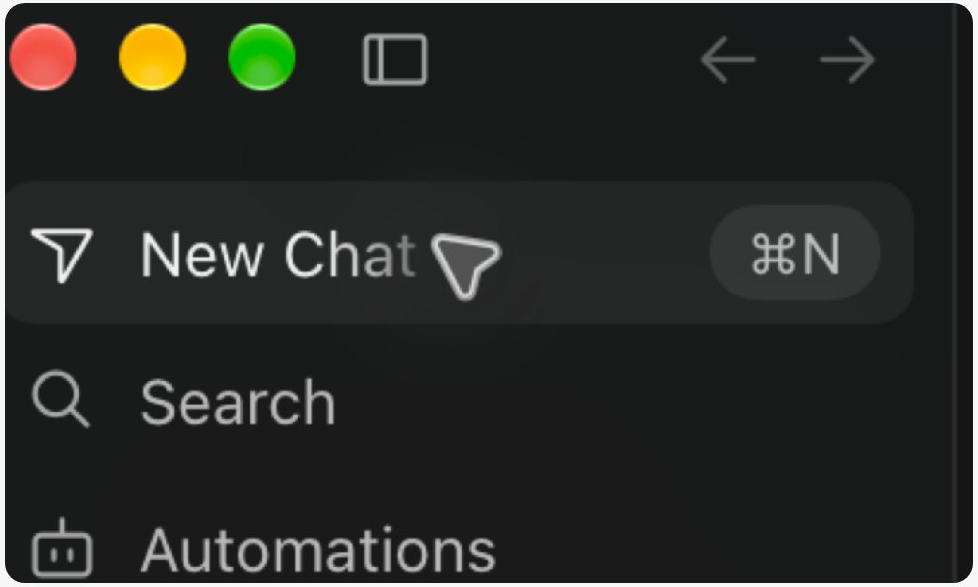
Open a new chat (1 min)

1. **Not the chat you're in — open a new one**

   The current chat still holds the context of what you just built

2. **A new chat doesn't have that context**

   It's the same situation as handing work to someone else



Skip this and the chapter doesn't work. The whole point is to create a “handing it to someone with no context” situation.

CHAPTER 3 FROM THE CLIENT

Two more requests came in

① When a pair matches, let the player flip again.

② From the third mistake onward, shorten the wait before the cards flip back to 0.5 seconds.

Neither of these is a **rule the AI would infer from what memory games usually do**. So not one of them is in what you just built.

CHAPTER 3 YOUR TURN

Have it implement the request (6 min)

Send the two items from the previous page as they are.

```
@session02/
① When a pair matches, let the player flip again.
② From the third mistake onward, the wait before the cards flip back is 0.5 seconds.
Don't change any other behaviour.
```

> **Send it in the new chat.** When the result comes back, **don't press Keep yet**. You count the lines in the diff on the next page.

CHAPTER 3 YOUR TURN

Write it down (3 min)

What to record	Answer
Lines changed (+/- in the diff)	
Did anything existing break	
Was it implemented correctly — and could you judge that yourself	

> **The third row is the most important question of the day.**

CHAPTER 3

How did you check that the third mistake onward really is 0.5 seconds?

Running it tells you nothing. Your eyes can't tell 0.5 seconds from 1 second.

To be sure, **you have to open the code and hunt for the number in the timer**.

CHAPTER 3

You opened a new chat a minute ago.

The AI in that chat **had no idea** what you were trying to build.

It read the code, **guessed**, and edited.

Whether eight cards was your decision or the AI's

Whether the one-second flip-back was intentional or an accident

None of it is written down, so whoever reads it can't know.

> **This time the one you handed it to happened to be an AI.** Tomorrow's you, or the person next to you, is in exactly the same position.

CURSOR PRACTICAL COURSE • SESSION 2 OF 5

CHAPTER 04 · 0:30 15 min

Change it freely

Add any feature you like to your memory game.

CH. 4 YOUR TURN

Add a feature you like (15 min)

1 Pick one feature (2 min)

e.g. a timer / different pictures (animals, flags, food) / difficulty / a clear animation / score by moves

2 Ask in a new chat (10 min)

Just `@session02/` then “Add (the feature).” **No need to be precise — that's today's way**

3 Run it and check (3 min)

Reopen it in the browser and see if the feature works. If you have time, add another one

CH. 4 COLUMN

Fixing the look by pointing at an element

Want to fix the look? You can **point** at it instead of describing it, with your game open in the built-in browser.

Screen	How
IDE view (the class screen)	Press the **Select Element** button in the built-in browser and click the element you want to change. Then say what to change in the chat
Agents Window	Press the **Design Mode** button in the browser. **Design Mode is only in the Agents Window's browser**

> Good for **look, spacing and layout**. **Not for calculation logic like scoring hands** (just ask Agent for that).

CURSOR PRACTICAL COURSE • SESSION 2 OF 5

CHAPTER 05 · 0:45 20 min

Presentations

Show everyone's games. Same 3 lines — how different are they?

CH. 5 PRESENTATIONS

How to present (3 min each)

Four people, 3 minutes each: 12 minutes. **We don't compare how finished it is.**

	What to talk about	Time
1	Run your memory game and show it	1 min
2	One thing that **appeared without being asked for**	30 s
3	Did the extra request from chapter 3 go in correctly — **could you check**?	30 s
4	The feature you added in chapter 4	1 min

> If it didn't run, show how far it got. That's enough.

CH. 5 COMPARE

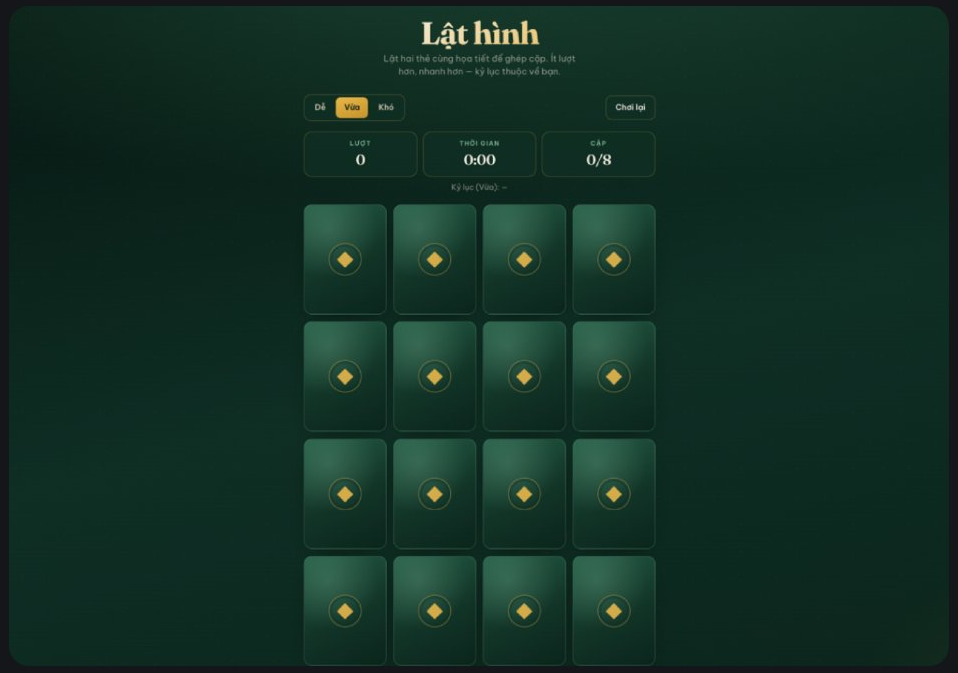
Compare everyone's memory games

What to compare	Person 1	Person 2	Person 3	Person 4
How many cards				
What's on the cards (numbers / emoji / colours)				
Is there a score or move counter				
Can you choose a difficulty				
Look (colours, layout)				

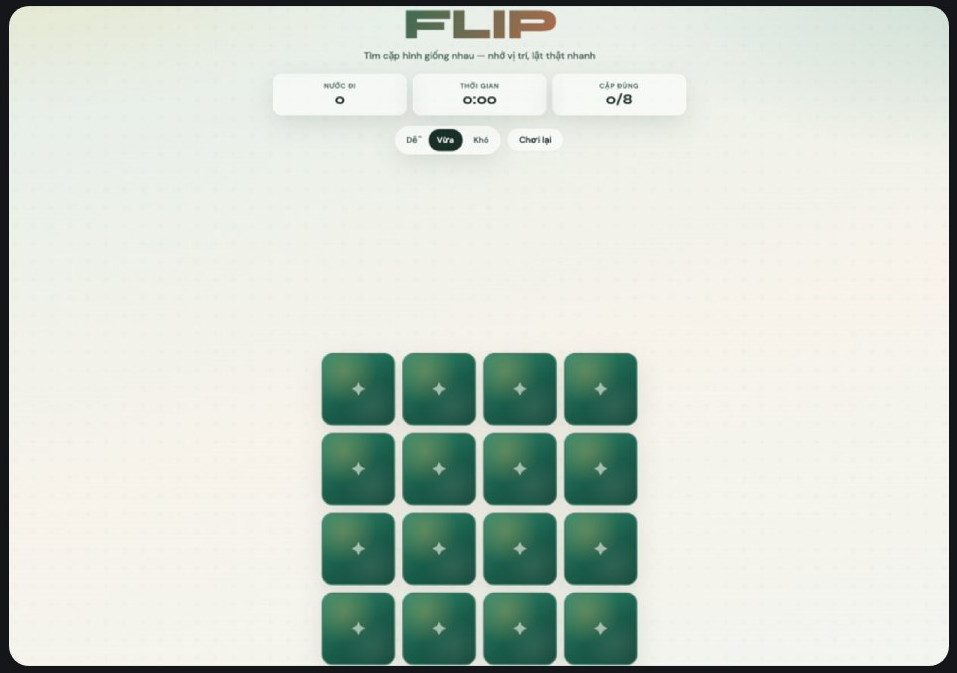
CHAPTER 5 ★

Same three lines, different results

Run 1



Run 2



> **The instructor sent the same 3 lines twice with the same settings.** These are different too.

CH. 5 SUMMARY

Same 3 lines, all different

Everyone made something different from the same request. That is vibe coding.

What happened	What it means
Everyone got something different	Even with the same request, **you get something different each time**
Things appeared that nobody asked for	You **can't tell what you asked for from what the AI decided**
You couldn't check the extra request went in	There is **no standard to judge** whether it's right

> **Don't criticise.** Every game runs. That is a real strength.

CURSOR PRACTICAL COURSE • SESSION 2 OF 5

CHAPTER 06 · 1:05 10 min

Summary

Put what we saw into words.

CH. 6 SUMMARY

What you handed over, what came back

Let's count what actually happened in chapter 2.

	Contents
What you handed over	**Three lines** (the game's name / HTML + JS / “make it nice”)
What you decided	That it's a memory game. That it runs in a browser. **That's all**
What came back	One playable game (in minutes)
What the AI decided	Card count・face design・layout・colours・score or no score・difficulty・how long until they flip back……

> The AI decided far more than you did.

CH. 6 SUMMARY

In one sentence

Vibe coding is fast.
But you can't build exactly what you intend.

“Fast” from chapter 2 is a strength. But there are three things you can't do.

Every run produced **something different**	→	**Can't aim**
You **can't tell** what the AI decided	→	**Can't hold it**
You **can't judge** whether it went in	→	**Can't verify**

Aim, know, and check. You can't do these three unless you decide things first.

CH. 6 SUMMARY

When to use which

When it's…	Use
Small, one-off, only you use it	**Vibe coding** (about memory-game size)
Many decisions, built with others, fixed later	**Spec-driven development** ← session 3

> **The line isn't size — it's how much there is to decide.** In session 3 we build a game with far more to decide (poker).

CURSOR PRACTICAL COURSE • SESSION 2 OF 5

CHAPTER 07 · 1:15 15 min

Review quiz and next time

Check sessions 1 and 2 with a few questions.

CH. 7 REVIEW QUIZ

Review quiz

One question at a time. **Write your answer on paper or in chat first**, then we check.

#	Question	Answer
1	What are the buttons that accept and reject the Agent's changes?	Keep / Undo
2	To hand a specific file to the Agent, what do you type in the input box?	@ (file name)
3	Which mode do you use when you only want to ask (no file changes)?	Ask
4	What do we call asking “make it nice” without deciding anything?	Vibe coding
5	Why couldn't you check on screen that “0.5 seconds” went in correctly?	Nobody decided “what counts as OK”
6	Why did everyone get something different from the same 3 lines?	The AI decided what we didn't

NEXT TIME • SESSION 3

Poker, built from requirements

A game with far more to decide than the memory game.

Asking “make it nice” won't get you the poker you intend.

We write the requirements first.

BEFORE NEXT TIME

Before next time (optional)

Both are optional. We check again at the start of session 3.

- [ ] Run `git pull` in `cursor-course/` (the Skills for session 3 come in)
- [ ] Read `05-prompting.md` (how to ask well)

Next time we build poker in a separate folder from today's memory game (`session02-spec/`).